

Docker Network

لما بنستعلم عن الـ Containers ← `docker inspect con6`

- في section خاص بـ Network

الـ container بيشتغل في Namespace منفصل

في Linux ، Docker بيعتمد على حاجة اسمها Namespaces

يعني كل Container بيبقى عنده:

- Process namespace → العمليات الخاصة بيه
- Network namespace → الشبكة الخاصة بيه
- File system namespace → ملفات خاصة بيه
- User namespace → المستخدمين

الـ Container بيبقى كأنه جهاز مستقل جوّه الجهاز الحقيقي.

Network Namespace

كل Container بيبقى عنده شبكة خاصة بيه : يعني

- Ip Address خاص بيه
- IP Address خاص بيه
- Interface خاص بيه
- Routing Table خاص بيه
- DNS خاص بيه

لما تستعلم عن الـ Container ← `docker inspect con6`

هتلاقي Section اسمه {.....}: "NetworkSettings"

- موجود فيه
- IPAddress
- Gateway
- Networks
- MacAddress

لو الـ Container ملوش Network

- مش هيعرف يكلم الـ Internet
- مش هيعرف يكلم الـ Containers
- مش هيعرف يكلم الـ Host نفسه

الـ Container كأنه جهاز افتراضي

لأنه فعلا :

- عنده OS environment خاص بيه (مش OS كامل زي VM , لكن Isolated environment)
- عنده شبكة خاصة بيه
- عنده processes مستقلة
- عنده IP خاص بيه

لكن الفرق:

- VM = OS كامل + Kernel خاص
- Container = نفس Kernel بتاع الـ Host

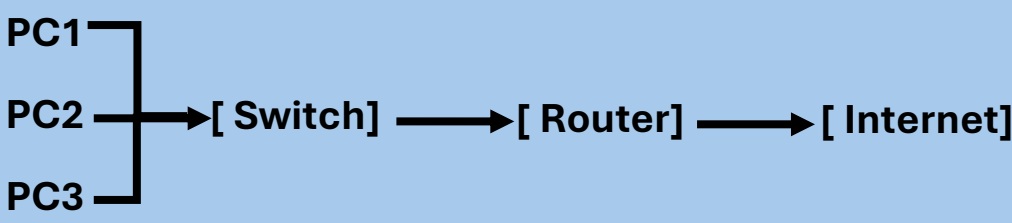
الفرق	
Containers	VM
نفس الـ Kernel	Kernel مستقل
بيئة معزولة	OS كامل
خفيف	ثقيل
سريع	بطئ

Bridge Network

شبكة افتراضية **Docker** يعملها على الـ **Host** وبيوصل بينها الـ **Containers** ببعض من غير ما يطلعوا على شبكة الشركة/البيت مباشرة , بيشفوا بعض بس من غير أي اتصال بالإنترنت الخارجي كأنه **Switch** افتراضي جوّه الجهاز

يعني إيه **Switch** ؟

جهاز (أو برنامج) يربط كذا جهاز مع بعض جوّه نفس الشبكة وبيخليهم يتكلموا مباشرة



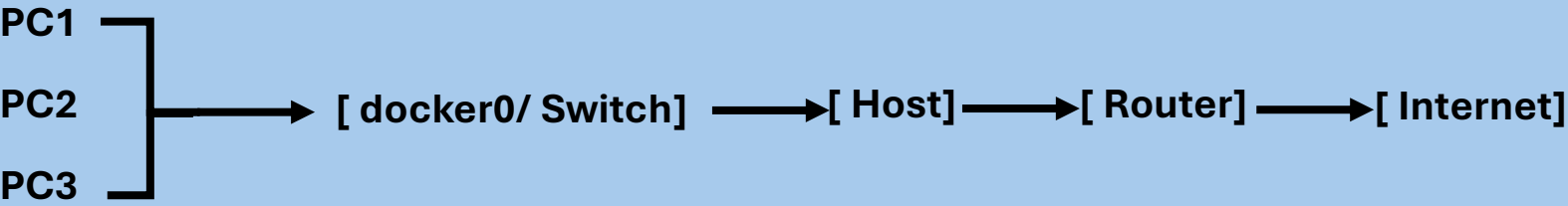
الـ **Docker** بيتعامل كأنه جهاز حقيقي فيه

يعني لازم:

- كارت شبكة
- IP
- شبكة

Docker يعمل:

- **Virtual Bridge** اسمه غالبًا : **docker0** كأنه **switch**
- كل: **Container**
 - بياخد **IP**
 - بيتوصل بالـ **bridge**
 - عنده **Interface** اسمه **eth0**



Port Mapping

ربط **Port** على الـ **Host** مع **Port** على الـ **Container** بشأن الـ **Container** معزول على شبكته وما حدث من بره يقدر يوصل له إلا لو فتحنا طريق.

الـ **Container** في **Network Namespace** معزول عنده **IP** داخلي: **172.17.0.2**
ده **IP** مش مرئي للشبكة الخارجية فـ **Docker** عمل حيلة ذكية

المكونات الأساسية للـ **Port Mapping**

1. **Network Namespace**

2. **Bridge (docker0)**

3. **veth pair**

4. **iptables (NAT)**

5. **Linux Kernel Networking**

1. اول ما يتم انشاء الـ **Container**

- **Docker** بيعمل **Network Namespace** جديدة

- **Interface** اسمه **eth0**

- **IP**

- **Gateway**

2. **Docker** يعمل **veth pair**

- زي سلك **Ethernet** له طرفين:

veth-container متوصل بـ **(eth0)** **Container**

veth-host → متوصل بـ **docker0**

3. **docker0 = Virtual Switch**

- **Bridge** على الـ **Host**

- بيجمع كل الـ **Containers**

4. لحظة الـ Port Mapping

المشكلة اللي Port Mapping بيحلّها

الـ Container في Docker:

- شغال في Network معزول
- ليه IP داخلي خاص بيه
- مش متشاف من برا إلا لو إنت سمحت بده

يعني لو عندك:

• تطبيق Web جوه Container

• شغال على Port 80

من المتصفح بتاع جهازك مش هتعرف توصله ليه؟

لأن الـ 80 ده جوه الكونتینر فقط.

وهنا بييجي دور Port Mapping

يعني إيه Port Mapping ؟

ربط Port على جهازك (Host) بـ Port جوه الكونتینر

بمعنى:

أي Request ييجي على Port معين في الجهاز

يتحول تلقائيًا لـ Port معين جوه الكونتینر

[Browser] → [http://localhost:8080] → [HOST] → [Port Mapping] → [CONTAINER] → [Port 80]

أي container واخد Network من الـ Bridge يبقى واخد Access على الـ Internet

كل الـ Container اللي على الـ Bridge هيكونوا شايفين بعض ودا منطقي جدا لأنهم على نفس الشبكة

لو عايز تختبر الـ Internet داخل الـ Container

```
yousef@DevOpsVM:~$ sudo docker exec -ti con5 bash
root@3befe6a65e8c6:/usr/local/apache2#
root@3befe6a65e8c6:/usr/local/apache2#
root@3befe6a65e8c6:/usr/local/apache2#
root@3befe6a65e8c6:/usr/local/apache2# ping google.com
PING google.com (142.251.209.78) 56(84) bytes of data.
64 bytes from ncmrsa-ai-in-f14.1e100.net (142.251.209.78): icmp_seq=1 ttl=127 time=48.2 ms
64 bytes from ncmrsa-ai-in-f14.1e100.net (142.251.209.78): icmp_seq=2 ttl=127 time=53.5 ms
64 bytes from ncmrsa-ai-in-f14.1e100.net (142.251.209.78): icmp_seq=3 ttl=127 time=47.4 ms
64 bytes from ncmrsa-ai-in-f14.1e100.net (142.251.209.78): icmp_seq=4 ttl=127 time=48.3 ms
64 bytes from ncmrsa-ai-in-f14.1e100.net (142.251.209.78): icmp_seq=5 ttl=127 time=44.9 ms
64 bytes from ncmrsa-ai-in-f14.1e100.net (142.251.209.78): icmp_seq=6 ttl=127 time=42.7 ms
```

اهداف الـ Docker Network

- يحصل (isolation) ما بين الـ Containers
- عشان اقدر اتحكم في الـ Container وأقول مين يخرج للـ Internet
- مين الـ User اللي برا الـ Network تدخل على الـ Container

هنعمل سيناريو يجمع أفكار كثيرة هنعمل Four Network اتنين internal واتنين external

a,b = external ----- x,y = internal

- create 4 Container اتنين في Network واتنين في Network مختلفة
- عايزين الاتنين اللي موجودين في الـ Network الـ external نفعلهم الـ Port Mapping
- الاتنين اللي في الـ Network الـ internal مش هنعلمهم Port Mapping
- نعمل a,b test شايفين بعض ولا لا وبرضو x,y شايفين بعض ولا لا
- نعمل a,b test طالعين على الـ Internet ولا لا وبرضو x,y طالعين على الـ Internet ولا لا
- عايزين a,b يشوفوا x,y من غير ما x,y يخرجوا للـ Internet

نعرض الـ Network الموجوده في الـ Docker

```
yousef@DevOpsVM:~$ sudo docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
813c4b3f4dc8        bridge             bridge              local
0949f201ad22        host               host                local
b30ea710d25c        none               null                local
```

- bridge = أي container متحددلوش شبكه بيدخل تلقائيا في الـ Bridge
- host = الـ container متصل بشبكة جهازك
- none = الـ container معزول 100% مناسب للمهام الخاصه والأمان

هنعمل Network Public

```
yousef@DevOpsVM:~$ sudo docker network create pub_net --subnet 10.0.0.0/24 --gateway 10.0.0.1
6228895e324f835dbc8f788d252f21a123973425938b38047441f5d1d5723425
```

pub_net = اسم الـ network

هنعمل Network Private

```
yousef@DevOpsVM:~$ sudo docker network create pri_net --subnet 10.11.0.0/24
--gateway 10.11.0.1 --internal
7bcc2dbb6f599ebd1213f735c36bda67fc7e47f6305e1ebdd85e98d225e0f4bd
```

pri_net = اسم الـ Network

--internal = بحدد انها private

كذا ال Internal True = Network private

```
yousef@DevOpsVM:~$ sudo docker network inspect pri_net
[
  {
    "Name": "pri_net",
    "Id": "7bcc2dbb6f599ebd1213f735c36bda67fc7e47f6305e1ebdd85e98d225e0f4bd",
    "Created": "2026-02-08T04:58:10.047645234+02:00",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "10.11.0.0/24",
          "IPRange": "",
          "Gateway": "10.11.0.1"
        }
      ]
    },
    "Internal": true,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    }
  }
]
```

كذا internal false = Network Public

```
yousef@DevOpsVM:~$ sudo docker network inspect pub_net
[
  {
    "Name": "pub_net",
    "Id": "6228895e324f835dbc8f788d252f21a123973425938b38047441f5d1d5723425",
    "Created": "2026-02-05T18:22:25.658446343+02:00",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "10.0.0.0/24",
          "IPRange": "",
          "Gateway": "10.0.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    }
  }
]
```

عملنا container وحددنا الـ Network بتاعته

```
yousef@DevOpsVM:~$ sudo docker run -d --name laba --network pub_net -p 90:80
--restart always --hostname laba myapp:v1
aa701c1c8f5b9c05606e30201511866a292096ac673094d616fe1f7f154d62b0
```

90:80 -p الـ 90 دا port انت بتحدده بيكون الخاص بالـ Host بتاعك ممكن تكتب أي رقم بشرط انه ميكونش محجوز عندك

الـ 80 في الـ Container ، التطبيق شغال على البورت 80 محتاج Docker يعرف ده.

myapp:v1 اسم الـ Image بتاعتك

تقدر تعرف جميع الـ Port اللي محجوزه في جهازك

```
yousef@DevOpsVM:~$ sudo ss -antwp
[sudo] password for yousef:
Netid      State      Recv-Q     Send-Q      Local Address:Port      Peer Address:Port
Process
icmp6      UNCONN     0           0            *:58                    *:.*
users:(( "NetworkManager",pid=783,fd=27))
tcp        LISTEN     0          4096         127.0.0.54:53           0.0.0.0:*
users:(( "systemd-resolve",pid=472,fd=17))
tcp        LISTEN     0          4096         127.0.0.53%lo:53       0.0.0.0:*
users:(( "systemd-resolve",pid=472,fd=15))
tcp        LISTEN     0          100         0.0.0.0:25             0.0.0.0:*
users:(( "master",pid=1660,fd=13))
tcp        LISTEN     0          4096         127.0.0.1:631          0.0.0.0:*
users:(( "cupsd",pid=1172,fd=7))
tcp        LISTEN     0          4096         [::1]:631             [::]:*
users:(( "cupsd",pid=1172,fd=6))
tcp        LISTEN     0          100         [::]:25               [::]:*
users:(( "master",pid=1660,fd=14))
tcp        LISTEN     0          4096         *:5000                 *:.*
users:(( "docker-registry",pid=1173,fd=3))
```

- كدا اتأكدنا port 90 مش محجوز ونقدر نشغل عليه

هنعمل عدد الـ Containers المطلوب لكل Network

```
yousef@DevOpsVM:~$ sudo docker run -d --name labb --network pub_net -p 91:80
--restart always --hostname labb myapp:v1
23d2518af5c88df278da52e297238f1332b9092089d492f9a5f5296bdefae4ff
```

هنعمل containers للـ private Network

```
yousef@DevOpsVM:~$ sudo docker run -d --name laby --network pri_net
--restart always --hostname laby myapp:v1
e5b92dc537e6f7f59529d53dc7041578f77eed1f8de2af5d0318c28ee2ead308
```

```
yousef@DevOpsVM:~$ sudo docker run -d --name labx --network pri_net
--restart always --hostname labx myapp:v1
fd1e07e4974b76e3d3e9dfbc5e6ed91ad7efd21cf1e69d22b49009a7b99237b1
```

عايزين نتأكد ان كل ال container تمام

```
yousef@DevOpsVM:~$ sudo docker inspect laba labb labx laby | grep -i ipa
      "IPAddress": "10.0.0.2",
      "IPAddress": "10.0.0.3",
      "IPAddress": "10.11.0.2",
      "IPAddress": "10.11.0.3",
```

عايزين نتأكد ان laba شايف labb

```
yousef@DevOpsVM:~$ sudo docker exec -ti labb bash
root@labb:/usr/local/apache2#
root@labb:/usr/local/apache2# ping -c2 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.121 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.045 ms

--- 10.0.0.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1059ms
rtt min/avg/max/mdev = 0.045/0.083/0.121/0.038 ms
root@labb:/usr/local/apache2#
```

- success

نجرب نبعت من lab الي labx

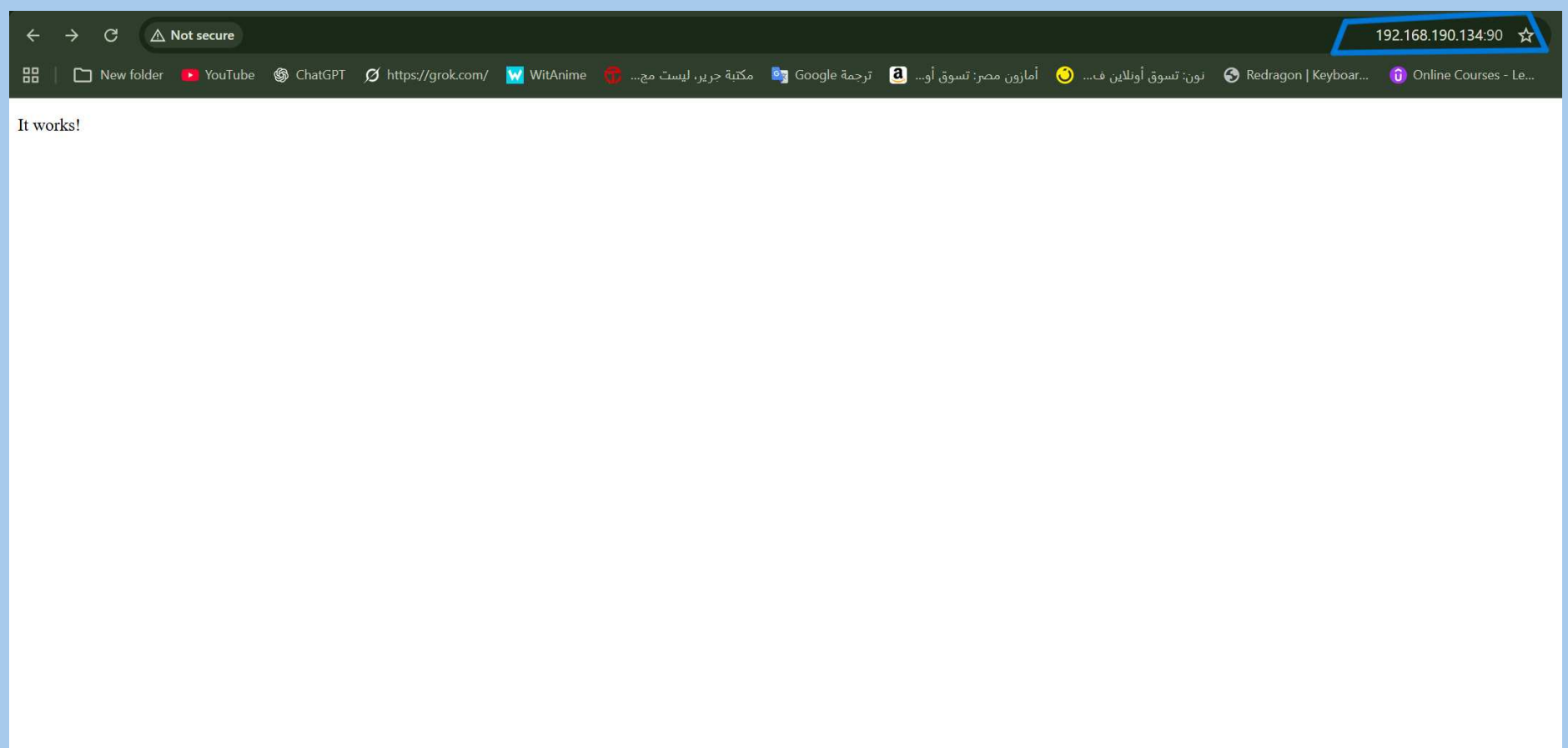
```
root@labb:/usr/local/apache2# ping -c2 10.11.0.2
PING 10.11.0.2 (10.11.0.2) 56(84) bytes of data.

--- 10.11.0.2 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1063ms

root@labb:/usr/local/apache2#
```

عايزين نتأكد ان ال Users الخارجيين يقدرُوا يدخلُوا على ال Container Public ولا لا

هنكتب في google == id machine ونكتب ال port



لو عايز اعمل تعديل على الصفحة دي واير كلمة it works

- هندخل جوا الـ container وهنلاقي Directory اسمه htdocs

```
yousef@DevOpsVM:~$ sudo docker exec -ti laba bash
[sudo] password for yousef:
root@laba:/usr/local/apache2#
root@laba:/usr/local/apache2#
root@laba:/usr/local/apache2# ls
bin  build  cgi-bin  conf  error htdocs  icons  include  logs  modules
root@laba:/usr/local/apache2#
```

- همدخل جوا ال Directory همدلاقي ملف اسمه index.html

```
root@laba:/usr/local/apache2# cd htdocs
root@laba:/usr/local/apache2/htdocs# ls
index.html
root@laba:/usr/local/apache2/htdocs#
```

بعد ما ندخل هنلاقي

```

<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html>
<head>
<title>It works! Apache httpd</title>
</head>
<body>
<p>It works!</p>
</body>
</html>

```

ويکدا تقدّر تعدل براحته

عايزين نخلى x,y يشوفوا a,b مع انهم على Network مختلفه

- هنخلي الـ containers نتصل بالشبكة الـ private

```
yousef@DevOpsVM:~$ sudo docker network connect pri_net labab
yousef@DevOpsVM:~$ sudo docker network connect pri_net labbb
```

عایزین نتأكد ان labb یقی شایف labx

```
yousef@DevOpsVM:~$ sudo docker exec -ti labb bash
root@labb:/usr/local/apache2#
root@labb:/usr/local/apache2# ping -c2 10.11.0.2
PING 10.11.0.2 (10.11.0.2) 56(84) bytes of data.
64 bytes from 10.11.0.2: icmp_seq=1 ttl=64 time=0.790 ms
64 bytes from 10.11.0.2: icmp_seq=2 ttl=64 time=0.100 ms

--- 10.11.0.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1008ms
rtt min/avg/max/mdev = 0.100/0.445/0.790/0.345 ms
root@labb:/usr/local/apache2#
```

لما نيجي نستعلم عن ال ip اللي شايفه بعض

```
yousef@DevOpsVM:~$ sudo docker inspect labb | grep -i ipadd
  "IPAddress": "10.11.0.5",
  "IPAddress": "10.0.0.3",
```

في خانه مهمه جدا في ال Network بتوضح ال containers المتصله بال Network

```
yousef@DevOpsVM:~$ sudo docker network inspect pub_net
```

```
    "Containers": {
      "23d2518af5c88df278da52e297238f1332b9092089d492f9a5f5296bdefae4ff": {
        "Name": "labb",
        "EndpointID": "0bfaefabf638b492a20d6ec7c8ec9e012812240b09a25e554859bfc3c79c2aa4",
        "MacAddress": "6a:63:5e:d6:29:ab",
        "IPv4Address": "10.0.0.3/24",
        "IPv6Address": ""
      },
      "aa701c1c8f5b9c05606e30201511866a292096ac673094d616fe1f7f154d62b0": {
        "Name": "laba",
        "EndpointID": "52ce430c1076768347da4e509bbf16debb1e52521820c3b045850c2ae5b04f20",
        "MacAddress": "a6:13:fe:c7:8b:de",
        "IPv4Address": "10.0.0.2/24",
        "IPv6Address": ""
      }
    }
  }
```

- هنلاقي خانه ال Containers فيها ال Containers المتصله بال Network المحدده

```
    "Containers": {
      "23d2518af5c88df278da52e297238f1332b9092089d492f9a5f5296bdefae4ff": {
        "Name": "labb",
        "EndpointID": "f0c0e9a5b5fd8e98c4f38206729ae6df423f0b125c9465b8ce7e57ece655fc76",
        "MacAddress": "ae:59:c6:a9:b9:79",
        "IPv4Address": "10.11.0.5/24",
        "IPv6Address": ""
      },
      "aa701c1c8f5b9c05606e30201511866a292096ac673094d616fe1f7f154d62b0": {
        "Name": "laba",
        "EndpointID": "ad4a494b7cb66ba1b42904f9fd11856e23ede5ce9ea80c4dbca86664712292a2",
        "MacAddress": "aa:47:50:76:d7:a5",
        "IPv4Address": "10.11.0.4/24",
        "IPv6Address": ""
      },
      "e5b92dc537e6f7f59529d53dc7041578f77eed1f8de2af5d0318c28ee2ead308": {
        "Name": "laby",
        "EndpointID": "b50800a4c286d4c68835d45713642efbe0e926ab4f2e9faf133eedbaed110a73",
        "MacAddress": "9e:7d:5c:5c:44:24",
        "IPv4Address": "10.11.0.3/24",
        "IPv6Address": ""
      },
      "fd1e07e4974b76e3d3e9dfbc5e6ed91ad7efd21cf1e69d22b49009a7b99237b1": {
        "Name": "labx",
        "EndpointID": "c460cb2dc071f29b5fad5493681e7c0c723b42904b1f12d25a8dae8e97344708",
        "MacAddress": "5a:8c:10:2b:8e:86",
        "IPv4Address": "10.11.0.2/24",
        "IPv6Address": ""
      }
    }
  }
```

- هنلاقي في ال private Network دي ال Containers المتصله بيها